Democracydot

Democracydot is an AI-supported civic technology project designed to make the legislative process more transparent, understandable, and engaging for the public. The goal is to turn dense policy and legal language into a clearer, more accessible understanding of what a Bill means, why it matters, and how it changes during parliamentary review.

How the project developed

Development followed the original goal of making the Bill process easier to understand and participate in:

1. **Set up the project.** Started with a Python and Streamlit prototype, established the project mission, and moved the work into this repository so development could continue here.
2. **Built the three-step workflow.** Organized the app around Bill Information, Democratic Engagement, and Democracy Index, with Tasmanian Parliament Bill lookup and OpenAI-supported summaries, questions, analysis, and comparisons.
3. **Added ways to participate and share results.** Added Billbow feedback, related reading and petition links, document uploads, and downloadable reports. The Democracy Index can display the compared document and an AI-assisted alignment estimate.
4. **Improved reliability and privacy.** Added a saved Parliament snapshot for hosting environments that cannot reach the live site, local SQLite with optional Postgres storage, a password-protected owner review page, and Privacy Policy and Terms pages. Name/mobile access is remembered on the user's device for later downloads.
5. **Refined the deployed experience.** Simplified navigation, made page behavior consistent, and applied the shared visual design and pink action-button styling.

This is a working prototype: some information is AI-generated or based on saved Parliament records, and users should check important details against the official sources.

Vision

Democracydot is intended to help users:

- Understand legislative proposals in plain language
- Compare proposed laws and amendments
- See likely impacts across communities and institutions
- Identify stakeholders, trade-offs, and policy implications
- Explore civic issues with AI-guided analysis rather than raw legal text alone

The longer-term aim is to support more informed public engagement around lawmaking by combining AI analysis with accessible presentation of policy information.

Technology stack

The project is currently grounded in a lightweight Python stack designed for experimentation and rapid iteration:

- Python
- Streamlit for an interactive user interface
- OpenAI models for policy analysis and summarization
- python-dotenv for environment management

Current implementation

The app opens on a public landing page. The three Bill pages are available without signing in, and the expanded sidebar provides navigation between them.

1. **Bill Information** — Search for a Tasmanian Bill or upload a document. The app can draft missing Bill details, run five focused analyses and a conclusion, and produce a Hansard summary, public-discussion summary, and combined HTML report. Essential-reading links include relevant Hansard and e-petition sources when available. Downloads on this page use a name/mobile form and a self-declared Facebook-follow tick; the Facebook declaration is not verified. A browser cookie remembers a completed name/mobile form on that device.
2. **Democratic Engagement** — Continue with the selected Bill to view available official documents and its parliamentary status. Users can ask Bill-grounded questions, request a proposed amendment, and submit Billbow feedback. The page can bundle the displayed official PDFs into a ZIP after the name/mobile form has been completed.
3. **Democracy Index** — Find the Bill and, when available, its Act as passed. Create an AI-assisted alignment estimate and a marked-up comparison document; the report can also show the importance weighting derived from related electronic petition signatures. The Index is an estimate, not an official parliamentary rating or legal advice.

The **Privacy Policy** (`/privacy`) and **Terms and Conditions** (`/terms`) are available as app pages. A notice on each page explains that using the app indicates acceptance of the terms.

The OpenAI features use `gpt-4o-mini`. The Democracy Index is an AI-assisted estimate based only on the supplied Bill texts; it is not an official parliamentary rating, a measure of democratic legitimacy, or legal advice.

Assessment criteria check

Checked against the assessment criteria (select a discrete legal task and build an AI tool that lets the user complete it).

Criterion	Status	Evidence
A specific legal task	Met	Analysing a Tasmanian Bill: its text, debates, public discussion, Act as passed, and how the Act differs from the Bill.
3 discrete steps	Met	The three pages: **Bill Information**, **Democratic Engagement** and **Democracy Index** (listed in `app.py`).
At least 5 AI calls	Met	Bill field auto-fill is 1 call, and the Guided Bill Analysis in `pages/bill_interaction.py` makes several more. A full Democracy Index adds 3 calls: public discussion, process analysis and document comparison. Hansard summary, public discussion, Q&A and amendment drafting each have their own call. The **Run full analysis** button on Bill Information makes 7 AI calls in one click.
User input	Met	Bill name search, uploads, questions, an amendment description and comments.
Output	Met	On-screen analyses, the Democracy Index score, a compare document and downloadable reports.
Deployed on Streamlit with an API credentials sidebar	Met	The sidebar in `app.py` has a password-type "OpenAI API key" field, falling back to `OPENAI_API_KEY` in secrets or the environment. Deployed at `democracydotcom.streamlit.app`.

Development setup

Create and activate the virtual environment, then install the project dependencies:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Enter your OpenAI API key in the **sidebar** ("OpenAI API key") when using the app; it is kept only in the session. Alternatively, put it in the root `.env` file or Streamlit secrets as a fallback:

```
OPENAI_API_KEY=your_api_key_here
```

Never commit or share your key. Text and documents are sent to OpenAI only when an AI feature is requested. Do not upload personal or sensitive information without permission.

To enable the private owner review page, set a strong password in the root `.env` file:

```
DEMOCRACYDOT_OWNER_PASSWORD=choose-a-strong-unique-password
```

For deployment, configure `DEMOCRACYDOT_OWNER_PASSWORD` in Streamlit secrets instead. Billbow comments and sign-up names and mobile numbers are stored in `comments.sqlite3` in the project root when running locally. On Streamlit Community Cloud, the local database file is not durable across restarts. Configure `DATABASE_URL` with a hosted Postgres connection string to persist sign-ups and comments between restarts.

To use a hosted Postgres database, copy its **connection string** (it looks like `postgresql://user:password@host/dbname?sslmode=require`) and add it to the app's Streamlit secrets. Keep the string private; never commit it.

```
OPENAI_API_KEY = "sk-..."
DEMOCRACYDOT_OWNER_PASSWORD = "choose-a-strong-password"
DATABASE_URL = "postgresql://user:password@host/dbname?sslmode=require"
```

The tables are created automatically the first time the app saves or reads data.

Run the app

After completing the development setup, start the Streamlit app from the repository root:

```
streamlit run app.py
```

Streamlit prints a local URL in the terminal. Open it in a browser to use the app and move between pages in the sidebar; no sign-in is required. Bill details and links are retained in the session. Comments and sign-ups use local SQLite unless `DATABASE_URL` is configured.

Deployment

The app is deployed from this repository on Streamlit Community Cloud (repository `mrogilvie/Democracydot.com`, branch `main`, main file `app.py`). Add `OPENAI_API_KEY` and `DEMOCRACYDOT_OWNER_PASSWORD` under **Advanced settings → Secrets**. Add `DATABASE_URL` there if durable hosted storage is configured. Pushes to `main` update the live app at `https://democracydotcom.streamlit.app/`.

Public landing website and installable PWA

A separate static website and installable Progressive Web App (PWA) are in `docs/`. To publish them, open the repository's **Settings → Pages**, choose **Deploy from a branch**, select `main` and the `/docs` folder, and save. GitHub Pages will publish the site at the URL shown in those settings; later pushes to `docs/` update it. Use HTTPS so browsers can install the PWA and cache the landing page for offline viewing. Visitors can install it from a supported browser; iPhone and iPad users can open the site in Safari and choose **Share → Add to Home Screen**. The installed app opens the landing page, which links to the existing Streamlit service; the interactive Bill tools still need an internet connection.

The landing page is served at the custom domain `https://democracydot.com` (the file `docs/CNAME` records it; do not delete it). The domain is registered with GoDaddy, with these DNS records:

Type	Name	Value
A	`@`	`185.199.108.153`, `185.199.109.153`, `185.199.110.153`, `185.199.111.153` (four records)
CNAME	`www`	`mrogilvie.github.io`

Any other `@` A records, such as GoDaddy's parking records, and domain forwarding must be removed. In **Settings** → **Pages**, the custom domain is `democracydot.com`; tick **Enforce HTTPS** once GitHub has finished issuing the certificate, which can take up to 24 hours.

QR code



`docs/qr-democracydot-app.png` is a QR code that opens the Streamlit app at `https://democracydotcom.streamlit.app/`. It is also published at `https://democracydot.com/qr-democracydot-app.png`. Keep the white border when printing it, at least about 2.5 cm across. To regenerate it, for example if the app address changes, run `python -m pip install "qrcode[pil]"` and then `qrcode.make("https://democracydotcom.streamlit.app/").save("docs/qr-democracydot-app.png")` in Python.

Facebook page branding

`brand/facebook/` holds images for the Democracydot Facebook page, made from the app's colours (navy `#102a43`, cornflower `#6495ed`, pink `#ff38d1`, paper `#f7f9ff`), Manrope and DM Sans fonts and three-bar logo:

- `democracydot-facebook-profile.png` (720 × 720): the profile picture. Facebook crops it to a circle, and the logo sits well inside that.
- `democracydot-facebook-cover.png` (1640 × 624): the cover photo. The text stays within the centre 1100 px so mobile cropping doesn't cut it, and clear of the bottom-left corner where the profile picture overlaps on desktop.

To change them, edit `profile.html` or `cover.html` in that folder and re-render each page at its image size with a headless browser (for example Playwright's `page.screenshot`).

Public repository

Streamlit Community Cloud allows only one private app per account, so this repository is made public. The code is safe to publish because it is a civic app with nothing sensitive in the source. Private data, such as sign-ups, comments, the OpenAI key, the owner password and any database connection string, stays out of the repository: keep secrets only in Streamlit **Secrets**, and never commit them.

Repository layout

```
app.py                      Streamlit page configuration and navigation
design_system.py            Shared Democracydot colors, typography, and component
styles
demodot.py                  Shared OpenAI, source-text, and Bill-session helpers
comment_store.py            SQLite (local) or Postgres (DATABASE_URL) storage for
Billbow submissions and sign-ups
bill_snapshot.py            Reads the saved Parliament Bill records in snapshot/
build_snapshot.py           Rebuilds snapshot/ (Bill records, status and PDFs) from
the live Parliament site
snapshot/                   Saved 2025-2026 Bill records and PDFs used when the live
site refuses the app
pages/
  home.py                   App overview and links into the three stages
  bill_interaction.py       Page 1, Bill Information: details, links, and optional
summary
  decision_phase.py         Page 2, Democratic Engagement: Billbow comments and Bill-
grounded Q&A
  bill_information.py       Page 3, Democracy Index: Bill comparison, markup, and
Demodot Meter
  privacy.py                Privacy Policy page, rendered from PRIVACY.md
  terms.py                  Terms and Conditions page, rendered from TERMS.md
  owner_review.py           Password-protected review of comments and sign-ups
PRIVACY.md                  Privacy Policy text
TERMS.md                    Terms and Conditions text
requirements.txt            Python dependencies
docs/                       Static public landing website (GitHub Pages)
  manifest.webmanifest      Installable PWA name, scope, display, and icons
  service-worker.js         Offline cache for the landing page
  install.js                Browser install prompt and iOS instructions
  qr-democracydot-app.png   QR code linking to the Streamlit app
```

Saved Bill snapshot

The Parliament of Tasmania website is behind Cloudflare bot protection, which can answer HTTP 403 to requests from cloud hosts such as Streamlit Community Cloud even though the same requests work from a browser. To keep Bill lookups working, the repository holds a snapshot of the Bills introduced in 2025 and 2026 in `snapshot/` (an `index.json` of Bill titles, document links, passage status and Act links, plus the PDFs in `snapshot/pdfs/`).

The app always tries the live site first. If Parliament refuses the request, it uses the snapshot for Bill matching, document links, status and PDFs, and shows a note with the snapshot date. After a refusal it skips live requests for five minutes so pages stay quick. Bills outside the snapshot still need the live site. Hansard, e-petition and AI web-search features are not part of the snapshot.

The snapshot goes out of date as Bills progress or new ones are introduced. There is no automatic refresh; a scheduled GitHub Action was tried and removed because Parliament's protection also refuses GitHub's servers, and live lookup is planned instead once Parliament IT allows access (see the roadmap). Until then, refresh it manually from a machine where Parliament is reachable (for example the Codespace), then commit `snapshot/`. The build stops without changing anything if Parliament returns no Bills:

```
python build_snapshot.py          # 2025 and 2026
python build_snapshot.py 2026     # chosen years only
```

Requesting live access from Parliament IT

Live lookups from the hosted app are refused by Parliament's bot protection, so the app relies on the snapshot. Parliament IT has been approached and is arranging access, which may take some time. The proposed request is kept here for reference:

> **Subject:** Request: allow read-only access to public Bills pages for the Democracydot civic app
>
> Hi [Name],
>
> Thanks for talking with me. This is the written follow-up to our conversation.
>
> **Background.** Democracydot (https://democracydot.com) is a civic-information app that helps the public understand Tasmanian Bills. It looks up Bills on the Parliament website, reads the Bill and Act documents, and presents plain-language summaries. The app is hosted on Streamlit Community Cloud.
>
> **The problem.** Requests from the app's hosting servers to www.parliament.tas.gov.au are refused with HTTP 403, apparently by the Cloudflare bot protection in front of the site. The same pages load fine from a normal browser and from other networks. Because of this, the app currently works from a saved copy of the Bills data, which is refreshed every few hours. It cannot show newly introduced Bills or status changes as they happen.
>
> **What I'm asking for.** Please consider allowing the app's requests through the bot protection. Any of these would work: 1. **Allow by identifying header (preferred).** The app would send a distinctive User-Agent, for example `DemocracydotBot/1.0 (+https://democracydot.com; contact: [email])`, and requests with it are allowed. Streamlit Community Cloud does not publish fixed IP addresses, so an IP allowlist is unreliable. 2. **Allow by IP.** If preferred, requests can be routed through a small relay with a

> fixed IP address, which I would give you. 3. **Another approach you prefer**, such as an official feed or API for Bills data, if one exists.
>
> **What the app requests.** - Only public, read-only pages: the Bills lists (`/bills/...`), individual Bill pages and their PDF documents. - Low volume: pages are fetched on demand when a user looks up a Bill, cached for an hour, and paced to avoid bursts. - No logins, forms, scraping of personal data, or other write actions. - Any rate limits or other conditions you set will be honoured.
>
> Please let me know what you need from me, such as the app's owner details, a technical contact, or a trial period. I am happy to adjust the app to meet your requirements.
>
> Kind regards, [Name] [Phone] | [Email] Democracydot

If IT chooses the header option, update `BROWSER_HEADERS` and `FALLBACK_HEADER_PROFILES` in `demodot.py` to send the agreed User-Agent.

Dependency list

The current dependency set is defined in `requirements.txt` and includes:

- openai
- streamlit
- markdown
- python-dotenv
- pypdf

Original feature instructions

The following is the user's original feature request, preserved verbatim as a product reference:

> This app will simplify constituents engagement with the legislative drafting and parliamentary debate process. It has three sub processes. Each sub process will have its own page on Streamlit. The app will help residents comprehend and participate in the legislative process. It will also allow them to share their views on the Bill and allow the app owner to review those comments.

Planned roadmap

1. **Live Bill lookup (planned patch).** Restore real-time lookups from the hosted app once Parliament IT has allowed Streamlit Community Cloud's requests (see "Requesting live access from Parliament IT"). Until then the app uses the saved snapshot. When access is in place, apply any agreed change (such as the allowed User-Agent in `demodot.py`), confirm live lookups work from the hosted app, and then review the snapshot notice.

2. Add moderation and retention controls for owner-reviewed Billbow submissions
3. Retrieve and cite official Bill texts and Hansard records
4. Improve AI analysis with source citations and traceable evidence
5. Add durable Bill records and support comparisons across Bills or policy areas